TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
BAN HỌC TẬP CÔNG NGHỆ PHẦN MỀM

ĐỀ THI THỬ SỐ 01
Môn: **Cấu trúc dữ liệu và giải thuật**
ĐÁP ÁN THAM KHẢO

Câu 1. (2 điểm)

a) Thế nào là một thuật toán sắp xếp ổn định? **Heap sort** và **Merge Sort** có phải thuật toán sắp xếp ổn định không? Cho biết độ phức tạp trung bình của 2 thuật toán sắp xếp theo kí hiệu Big-O

Thuật toán sắp xếp ổn định là thuật toán giữ lại vị trí tương đối của các phần tử có giá trị bằng nhau.
Heap sort không phải thuật toán sắp xếp ổn định.
Merge sort là thuật toán sắp xếp ổn định.
Độ phức tạp của cả hai thuật toán theo kí hiệu Big-O là $O(N \log N)$ với N là số lượng phần tử của dãy.

b) Hãy cài đặt bằng ngôn ngữ C/C++ một trong 2 thuật toán trên.

Thuật toán Merge sort

```cpp
#include <bits/stdc++.h>
using namespace std;
const int lim = 1e6;
int a[lim];

void Input(int a[],int& n)
{
    cin>>n;
    for (int i=1;i<=n;i++) cin>>a[i];
}

void Merge(int a[],int l,int mid,int r)
{
    int i = l, j = mid+1;
    vector<int> vt(r-l+10);
    int pos =l;
    while(i<=mid && j<=r)
    {
        if (a[i]<=a[j])
        {
            vt[pos++] = a[i++];
        } else vt[pos++] = a[j++];
    }
    while(i<=mid)
    {
        vt[pos++] = a[i++];
    }
```

```
    while(j<=r)
    {
        vt[pos++] = a[j++];
    }
    for (int k=l;k<=r;k++)
    {
        a[k]=vt[k];
    }
}
void Merge_Sort(int a[],int n,int l,int r)
{
    if (l==r) return ;
    int mid = (l+r)/2;
    Merge_Sort(a,n,l,mid);
    Merge_Sort(a,n,mid+1,r);
    Merge(a,l,mid,r);
}

void Output(int a[],int n)
{
    for (int i=1;i<=n;i++) cout<<a[i]<<" ";
    cout<<endl;
}
int main()
{
    int n;
    Input(a,n);
    Merge_Sort(a,n,1,n);
    Output(a,n);
}
```

Thuật toán Heap sort:

```
#include <bits/stdc++.h>
using namespace std;
#define sz 1000006
int a[sz];
void Input(int a[],int& n)
{
    cin>>n;
    for (int i=1;i<=n;i++) cin>>a[i];
}
void Heapify(int a[], int n, int index)
{
    int max_val = index;
    int left = 2 * index;
    int right = 2 * index + 1;
    if (left <= n && a[left] > a[max_val]) max_val = left;
    if (right <= n && a[right] > a[max_val]) max_val = right;
```

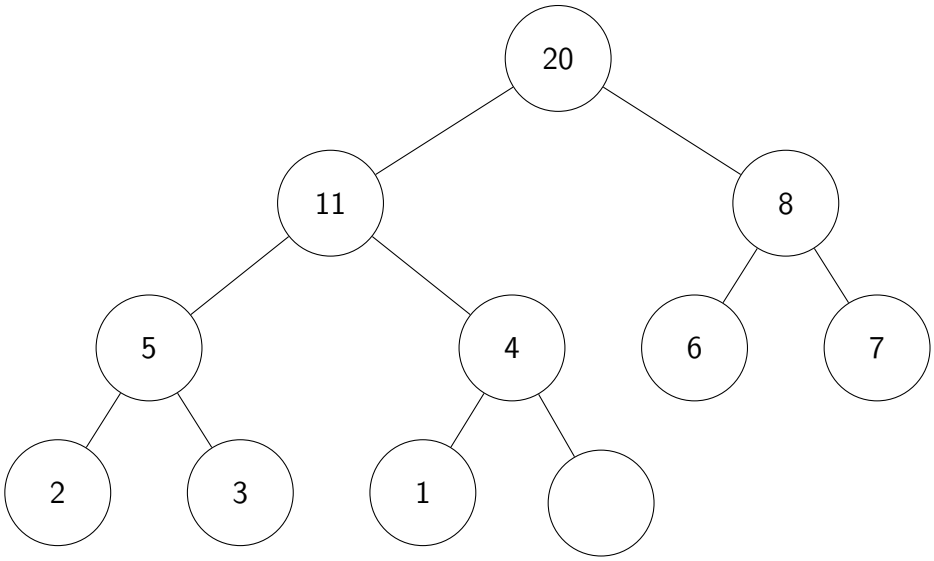
```
    if (max_val != index)
    {
        swap(a[index], a[max_val]);
        Heapify(a, n, max_val);
    }
}
// index = 0  ===> n/2-1
// index =1  ===> n/2
void Build(int a[],int n)
{
    for (int i=n/2;i>=1;i--)
    {
        Heapify(a,n,i);
    }

}
// build 1 cay

void Heap_Sort(int a[],int n)
{
    Build(a,n);
    int kc = n;
    for (int i=n;i>=1;i--)
    {
        swap(a[1],a[kc--]);
        Heapify(a,kc,1);
    }
}
void Output(int a[],int n)
{
    for (int i=1;i<=n;i++)
    {
        cout<<a[i]<<" ";
    }
    cout<<endl;
}
int main()
{
    int n;
    Input(a,n);
    Heap_Sort(a,n);
    Output(a,n);

}
```

c) Hãy vẽ cây Heap sau khi sắp xếp dãy tăng dần (đã được thêm vào theo thứ từ từ trái sang phải): 11 5 8 20 4 6 7 2 3 1



Câu 2. (2 điểm) Một thành phố nọ đang thực hiện việc phân phát lương thực cho toàn bộ người dân trong thành phố. Để tiện lợi cho việc phân phát, mỗi người dân sẽ được đánh số thứ tự từ 1 đến P $(1 \leq P \leq 10^9)$ và đứng theo hàng theo đúng thứ tự trên. Quy trình phân phát sẽ được diễn ra như sau: Có C lệnh $(1 \leq C \leq 1000)$, khi một lệnh "N"được gọi lên, người đứng ở đầu hàng sẽ tiến đến và nhận lương thực, sau đó quay trở về cuối hàng.
Việc phân phát sẽ dừng lại khi hết lệnh. Tuy vậy, có những quý tộc được hưởng quyền lợi đặc biệt: Khi một lệnh "E x"kêu lên, người mang số thứ tự x sẽ bước lên đầu hàng đứng. Quản lý thành phố cần biết người mang số thứ tự nào sẽ nhận được lương thực mỗi khi lệnh "N"được kêu lên. Bạn hãy giải quyết bài toán cho quản lý nhé!

Input

- Dòng đầu tiên chứa số lượng bộ test t $(1 \leq t \leq 10)$
- Dòng thứ hai chứa 2 số nguyên P và C - lần lượt là số lượng người và số lệnh được thực hiện
- Trong C dòng tiếp theo sẽ có các lệnh theo 1 trong 2 dạng:
  - N: Kêu người ở đầu hàng nhận lương thực.
  - E x: Kêu người có số thứ tự x lên đầu hàng.

Output

- Dòng đầu tiên in ra "Case #(số thứ tự của case):".
- Với mỗi lệnh N hãy xuất ra một số nguyên - là STT của người đứng ở đầu hàng sẽ lấy lương thực.

Input	Output
2 3 6 N N E 1 N N N 10 2 N N	Case #1 1 2 1 3 2 Case #2 1 2

a) Hãy cho biết Cấu trúc dữ liệu phù hợp để giải quyết bài toán trên

Cấu trúc dữ liệu hàng đợi (queue)

b) Cài đặt thuật toán dựa trên cấu trúc dữ liệu của câu a) với thời gian chạy dưới 1s, giả sử 1s máy tính chạy được 10^6 phép tính.

Ý tưởng thuật toán:

- Do số lượng câu lệnh C ($C \leq 10^3$) nên tối đa chỉ có C người nhận lương thực.
- Ban đầu tạo một hàng đợi lưu trữ 1000 người đầu tiên:
  - Với lệnh `N` $\Rightarrow$ Xuất ra người đầu của queue.
  - Với lệnh `E` x $\Rightarrow$ Tạo thêm 1 hàng đợi để lưu trữ x và các người trong hàng đợi ban đầu.

Độ phức tạp: $O(t * C^2) = O(10 * 1000^2) = O(10^7)$

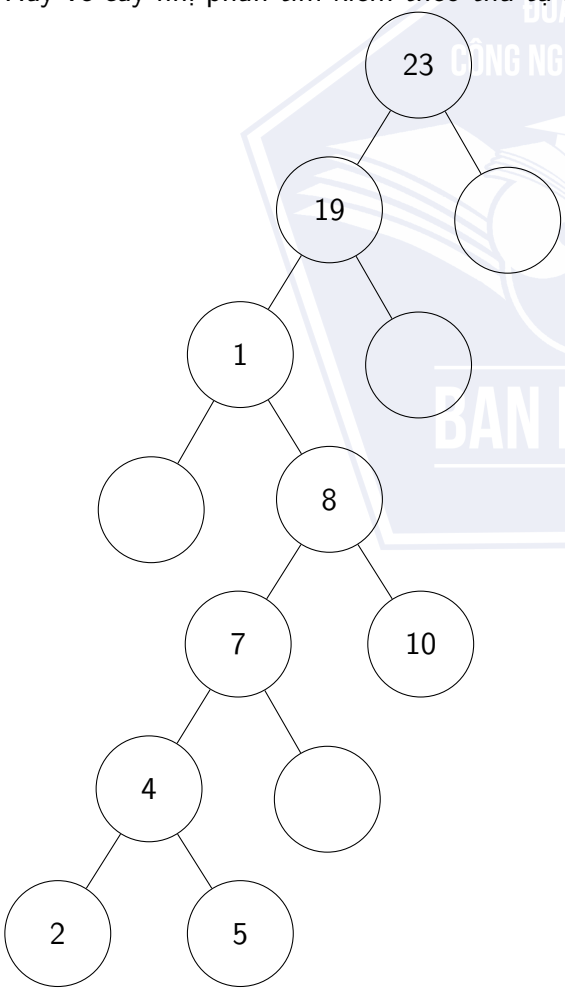
```cpp
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int tc;
    cin>>tc;
    for (int k=1;k<=tc;k++)
    {
        cout<<"Case #"<<k<<": "<<endl;
        int P, C;
        cin>>P>>C;
        queue<int> Q1;
        for (int i=1;i<=C;i++) Q1.push(i);
        for (int i=1;i<=C;i++)
        {
            char order;
            cin>>order;
            if (order=='N')
            {
                cout<<Q1.front()<<endl;
                Q1.push(Q1.front());
                Q1.pop();
            } else
            {
                int x;
                cin>>x;
                queue<int> Q2;
                Q2.push(x);
                while(!Q1.empty())
                {
                    if (Q1.front()!=x)
```

```
                        {
                            Q2.push(Q1.front());
                        }
                        Q1.pop();
                    }
                    while(!Q2.empty())
                    {
                        Q1.push(Q2.front());
                        Q2.pop();
                    }
                }
            }
        }
    }
```

Câu 3. (2.5 điểm) Cho dãy số sau a = {2, 5, 10, 4, 7, 8, 1, 19, 23} (*)

a) Hãy vẽ cây nhị phân tìm kiếm theo thứ tự nhập vào từ phải sang trái của dãy số a.



b) Hãy cho biết phải thực hiện bao nhiêu phép so sánh để tìm được node 10 và 4.

Node 10: 3
Node 4: 6

c) Hãy cài đặt hàm đếm số lượng node có maxleft $\geq$ minrightdựa theo *template* có sẵn dưới đây: (dãy * ban đầu, các số được nhập từ trái qua phải, kết thúc khi gặp số −1)

```
#include <bits/stdc++.h>
using namespace std;

struct NODE
{
        int key;
        NODE* left;
        NODE* right;
};
typedef NODE* TREE;

NODE* makenode(int x)
{
        NODE* p = new NODE;
        p->key = x;
        p->left = NULL;
        p->right = NULL;
        return p;
}

// INSERT CODE HERE
void them(TREE& t,int x)
{
    TREE p = makenode(x);
    if (t==NULL)
    {
        t = p;
        return;
    }
    if (t->key<x) them(t->right,x);
    else them(t->left,x);
}
void CreateTree(TREE& t)
{
    t = NULL;
    int x;
    cin>>x;
    while(x!=-1)
    {
        them (t,x);
        cin>>x;
    }
}
int ans =0;
int getleft(TREE t )
{
    if (t==NULL)
    {
```

```
        return INT_MIN;
    }
    int left = getleft(t->left);
    int right = getleft(t->right);
    return max({t->key,left,right});
}

int getright(TREE t )
{
    if (t==NULL)
    {
        return INT_MAX;
    }
    int left = getright(t->left);
    int right = getright(t->right);
    return min({t->key,left,right});
}
void Duyet(TREE t)
{
    if (t==NULL)
    {
        return;
    }
    int left = getleft(t->left);
    int right = getright(t->right);
    if (left>=right) ans++;
    Duyet(t->left);
    Duyet(t->right);
}
void Function(TREE t)
{
    Duyet(t);
    cout<<"Number of nodes having maxleft >= minright: "<<ans<<endl;
}


int main()
{
       TREE t;
       CreateTree(t);
       Function(t);
       return 0;
}
```

Câu 4. (2 điểm) Cho bảng băm H có kích thước 13 và các khóa như sau

0	1	2	3	4	5	6	7	8	9	10	11	12
NULL	NULL	28	NULL	NULL	NULL	NULL	NULL	47	NULL	23	NULL	NULL

Cho tập hợp số nguyên a = {49 , 26, 13 , 5 , 8}.

Cho 2 hàm băm sau: $h(key) = key \,\%\, M$ và $g(key) = 7 - (key \,\%\, 7)$

Hàm băm kép: $f = (h(key) + i \times g(key)) \,\%\, M$ *(M là kích thước bảng băm)*

a) Hãy vẽ lại bảng băm H khi nhập lần lượt từ trái qua phải tập hợp a bằng hàm băm $h(key)$. Nếu xảy ra đụng độ hãy sử dụng hàm băm kép.
Cho biết có tồn tại khóa nào không thể băm vào bảng? Nếu có hãy liệt kê theo thứ tự nhập vào.

Giá trị các key khi hash các giá trị từ dãy:

$$\begin{aligned} h(26) &= 0 \\ h(13) &= 1 \\ h(49) &= 4 \\ h(5) &= 5 \\ h(8) &= 9 \end{aligned}$$

Bảng băm H:

0	1	2	3	4	5	6	7	8	9	10	11	12
26	13	28	NULL	49	3	NULL	NULL	47	8	23	NULL	NULL

b) Hãy trình bày các bước tìm khóa 13.

Sử dụng hàm băm $h(key) = key\%m$

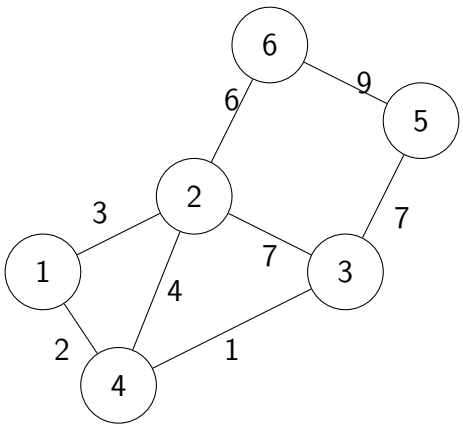
$$h(13) = 13\%13 = 0$$

Tại ô 0 không phải 13 $\Rightarrow$ Đụng độ
Sử dụng hàm băm kép:

$$\begin{aligned} f &= (h(key) + i * g(key)\%M) \\ &= (0 + i * (7 - (13\%7)))\%M \\ &= i\%13 \end{aligned}$$

- Tại $i = 1 \Rightarrow f = 1$. Tại ô 1 có giá trị 13 $\Rightarrow$ Tìm thấy

Câu 5. (1.5 điểm). Cho đồ thị vô hướng $G = (V, E)$



a) Hãy biểu diễn đồ thị bằng ma trận kề.

Ma trận kề:

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	1	0	1	1	0	1
3	0	1	0	1	1	0
4	1	1	1	0	0	0
5	0	0	1	0	0	1
6	0	1	0	0	1	0

b) Hãy cho biết kết quả duyệt đồ thị khi sử dụng thuật toán DFS với đỉnh bắt đầu là đỉnh 3.

Thuật toán DFS:

p	Γ(p)	Open	Close
		3	
3	2 4 5	5 4 2	3
5	3 6	6 4 2	3 5
6	2 5	4 2	3 5 6
4	1 2 3	1 2	3 5 6 4
1	2 4	2	3 5 6 4 1
2	1 3 4		3 5 6 4 1 2

Kết quả duyệt đồ thị: 3 5 6 4 1 2

c) Sử dụng thuật toán dijkstra tìm đường đi ngắn nhất từ đỉnh 5 đến đỉnh 1.

p	Γ(p)	Open	Close
		5	
5_0	3 6	(3, 7), (6, 9)	5
3_7	2 4 5	(2, 14), (4, 8), (6, 9)	5 3
4_8	1 2 3	(1, 10), (2, 12), (6, 9)	5 3 4
6_9	2 5	(1, 10), (2, 12)	5 3 4 6
1_{10}			

Vậy đường đi ngắn nhất từ 5 $\rightarrow$ 1 là 5 3 4 1
Trọng số: 10